MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY

BACHELOR OF INFORMATION SYSTEMS AND TECHNOLOGY (BIST)
EVENING

SYSTEM INTEGRATION AND DEPLOYMENT (IST 3206)

LECTURER: MR. MUCHAKE BRAIN

SRS FOR HOSTEL BOOKING SYSTEM(IEEE STD 830-1998 COMPLIANT)

NAME	STUDENT NUMBER	REGISTRATION NUMBER
MUGISHA RAYAN	2300724192	23/U/24192/EVE
AKULE ROBERT	2300705785	23/U/O5785/EVE
MUSIIME MARTHA TRISHA	2300724223	23/U/24223/PS
AINEBYOONA EVANS	2300725474	23/U/25474/EVE
SSESSANGA NEHEMIAH	2300717751	23/U/17751/PS

Date: 4th May, 2026

Status :Final Submission

Table of Contents

1 Introduction

1.1 Purpose ----------2
1.2 Scope ----------2
1.3 Definitions, Acronyms and Abbreviations----------2
1.4 References ----------3
1.5 Overview ----------4

2 Overall Description

2.1 Product Perspective ----------4
2.2 Product Functions ----------5
2.3 User Characteristics ----------6
2.4 Constraints ----------6
2.5 Assumptions and Dependencies ----------6

3 Specific Requirements

3.1 Functional Requirements ----------7
3.2 Non-Functional Requirements ----------11
3.3 External Interface Requirements ----------13
3.4 System Constraints ----------14

4 Appendices

4.1 Appendix A - Glossary ----------15
4.2 Appendix B - Use Case Summary ----------15

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Hostel Booking System, a full-stack web application . It is prepared in accordance with the IEEE Std 830-1998 Recommended Practice for Software Requirements Specifications.

This document is intended for the following audiences:

- Software developers and engineers building the system
- Testers validating system behaviour against stated requirements
- Future maintainers and stakeholders who may extend the system

1.2 Scope

The Hostel Booking System is a secure, web-based platform that replaces the current manual, cash-based hostel booking process at Makerere University. The system shall enable students to discover, book, and pay for hostel accommodation online, and shall enable hostel custodians to manage bookings, room inventory, and payments through a dedicated dashboard.

The system shall NOT:

- Manage academic records or student registration
- Process physical cash payments
- Handle hostel construction or property management activities
- Integrate with the Makerere University ERP system in this version

1.3 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
IEEE	Institute of Electrical and Electronics Engineers
MERN	MongoDB, Express.js, React, Node.js - a full-stack JavaScript framework

API	Application Programming Interface
REST	Representational State Transfer - an architectural style for APIs
JWT	JSON Web Token - a compact token used for secure authentication
CORS	Cross-Origin Resource Sharing - a browser security mechanism
SPA	Single Page Application
ODM	Object Document Mapper (Mongoose for MongoDB)
MVC	Model-View-Controller - a software design pattern
UGX	Ugandan Shilling - the currency used for hostel payments
Student	A registered Makerere University student seeking hostel accommodation
Custodian	A hostel manager responsible for approving bookings and managing rooms
Admin	A platform administrator with system-wide oversight privileges
Booking	A formal request by a student to reserve a hostel room for a semester
Receipt	A digitally generated PDF document confirming successful payment

1.4 References

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.

[2] ISO/IEC/IEEE 29148:2011, Systems and Software Engineering - Life Cycle Processes - Requirements Engineering.

[3] BIST Group 3, Hostel Booking System – Development Plan (Assignment 1), Makerere University, 2025.

[4] BIST Group 3, System Integration and Deployment – Assignment 1 (IST 3206), Makerere University, 2025.

[5] MongoDB Documentation – https://www.mongodb.com/docs/

[6] Express.js Documentation – https://expressjs.com/

[7] React Documentation – https://react.dev/

[8] Render.com Deployment Documentation – https://render.com/docs

1.5 Overview

The remainder of this SRS is organised as follows:

- Section 2 - Overall Description: Describes the general context, product perspective, major functions, user characteristics, constraints, and assumptions.
- Section 3 -Specific Requirements: Provides detailed functional, non-functional, and interface requirements using the IEEE 'shall' notation. Each requirement is uniquely identified for traceability.
- Section 4 - Appendices: Contains the glossary and a use case summary table.

2 Overall Description

2.1 Product Perspective

The Hostel Booking System is a new, self-contained web application that operates independently from existing Makerere University systems. It is not a replacement module for a larger ERP; it is a purpose-built platform for hostel accommodation management.

The system consists of three integrated tiers:

- Client Layer: A React 19 + Vite Single Page Application (SPA) served from a global CDN on Render.com. Users access the system via a web browser on any device.
- Application Layer: A Node.js + Express.js RESTful API server hosted on Render.com (Oregon, USA). It handles all business logic, authentication, and data processing.
- Data Layer: A MongoDB Atlas NoSQL cloud database cluster that stores all system data, including users, hostels, rooms, bookings, payments, and reviews.

The system communicates with the following external services:

External Service	Purpose

Mobile Money (Airtel/MTN)	Processing student hostel payments
Visa / MasterCard Gateway	Processing credit/debit card payments
Bank Transfer Service	Processing direct bank payment confirmations
html2canvas + jsPDF	Client-side PDF receipt generation
Google Maps API	Displaying interactive hostel location maps
Email Service (SMTP)	Sending booking confirmations and receipts to students

2.2 Product Functions

The following are the major functions the Hostel Booking System shall provide:

User Authentication: Secure registration and login using email/password with JWT session management.

Hostel Discovery: Search and filter hostels by name, location, price, gender, and room type.

Room Booking: Multi-step booking form capturing personal, academic, and next-of-kin information.

Payment Processing: Secure payments via Mobile Money, Credit/Debit Card, or Bank Transfer.

Receipt Generation: Automatic downloadable PDF receipt upon successful payment.

Booking Management: Custodian dashboard for reviewing, approving, or rejecting booking requests.

Room Inventory Management: Visual floor plan with real-time room status (Available / Partial / Booked).

Maintenance Requests: Students submit maintenance issues; custodians update resolution status.

Student Dashboard: Personal booking history, status tracking, and profile management.

2.3 User Characteristics

The system serves three distinct user classes:

User Class	Technical Level	Primary Responsibilities
Student	Low to Moderate - basic smartphone/PC literacy	Search hostels, book rooms, make payments, download receipts, track booking status

Custodian	Moderate - comfortable with web dashboards	Manage booking requests, approve/reject, assign rooms, update inventory
Administrator	High - full system access	Manage all users, hostels, and system configuration; generate platform reports

2.4 Constraints

The following constraints apply to the development and operation of the system:

Technology Stack: The system must be built using the MERN stack (MongoDB, Express.js, React, Node.js).

Deployment Platform: The system must be deployed on Render.com using two separate services (static frontend + Node.js backend).

Authentication: The system must use JWT-based stateless authentication; no server-side sessions shall be used.

Database: All persistent data must be stored in MongoDB Atlas; no relational databases are permitted.

Security: All passwords must be hashed using bcryptjs before storage; plain-text passwords are prohibited.

Payment Currency: All prices and transactions must be expressed in Ugandan Shillings (UGX).

Browser Support: The frontend must support modern browsers (Chrome, Firefox, Edge, Safari-latest 2 versions).

Internet Dependency: The system requires an active internet connection; offline functionality is out of scope.

2.5 Assumptions and Dependencies

Assumptions:

- All students using the system have a valid Makerere University registration number.
- Hostel custodians have been registered in the system by the administrator before going live.

- Students have access to a smartphone or computer with a modern web browser.
- Mobile Money services (Airtel Money and MTN MoMo) are available and accessible in Uganda.

Dependencies:

- The system depends on the availability of MongoDB Atlas cloud services.
- PDF receipt generation depends on the html2canvas and jsPDF JavaScript libraries.
- Payment processing depends on third-party Mobile Money and card payment gateway uptime.
- Map functionality depends on Google Maps API availability and quota limits.

3 Specific Requirements

This section specifies all software requirements to a level of detail sufficient to enable designers to design a system satisfying those requirements, and testers to verify that the system satisfies them. Each requirement is uniquely identified using the prefix FR (Functional), NFR (Non-Functional), or EIR (External Interface), followed by a sequential number.

3.1 Functional Requirements

3.1.1 Authentication and User Management

FR-001

The system shall allow new users to register by providing a full name, email address, password, student/registration number, and role (student or custodian).

FR-002

The system shall validate email uniqueness during registration and reject duplicate email addresses with an appropriate error message.

FR-003

The system shall hash all user passwords using bcryptjs with a minimum salt round of 10 before persisting to the database.

FR-004

The system shall authenticate users by verifying their email and password credentials and issue a signed JWT upon successful login.

FR-005

The system shall enforce JWT expiry of 30 days; expired tokens shall be rejected and the user shall be prompted to log in again.

FR-006

The system shall allow authenticated users to view and update their profile information (name, phone number, profile photo).

FR-007

The system shall restrict access to protected API routes by verifying the Bearer token in the Authorization header before processing any request .

3.1.2 Hostel Discovery and Search

FR-008

The system shall display a searchable list of all registered hostels on the public home page without requiring login.

FR-009

The system shall allow users to filter hostels by at least the following criteria: location/campus, price range, gender preference (male/female/unisex), and room type.

FR-010

The system shall display a detailed hostel profile page showing: hostel name, location, photos, pricing per semester, amenities, available room types, star rating, and an interactive Google Maps embed.

FR-011

The system shall display the real-time availability status of each room type (Available, Partially Booked, Fully Booked) on the hostel detail page.

3.1.3 Room Booking

FR-012

The system shall prevent a student from initiating a new booking if they already have an active booking (status: Pending or Confirmed) in the system.

FR-013

The system shall present a multi-step booking form collecting: (a) personal information - full name, student number, college, phone; (b) next-of-kin information - full name, phone, email, relationship;

(c) health status declaration; (d) required document uploads (Student ID, Payment Receipt, University Admission Form, Medical Form).

FR-014

The system shall require the student to check an acknowledgement checkbox confirming they agree to all hostel rules and regulations before submitting the booking form.

FR-015

The system shall create a booking record in the database with status Pending Approval immediately after a student submits a valid booking form.

FR-016

The system shall generate a unique booking reference token and associate it with each booking record at the time of creation.

3.1.4 Payment Processing

FR-017

The system shall present the student with a payment page showing a Booking Summary (hostel name, room type, duration, room fee, security fee, and total in UGX) before processing payment.

FR-018

The system shall support at least three payment methods: (a) Mobile Money via Airtel or MTN; (b) Credit/Debit Card via Visa or MasterCard; (c) Direct Bank Transfer.

FR-019

The system shall display a secure payment badge confirming that payment information is encrypted before the student submits payment details.

FR-020

The system shall generate a downloadable PDF receipt automatically upon successful payment confirmation.

FR-021

The system shall include the following information in the PDF receipt: booking reference, student name, student number, hostel name, room type, semester, amount paid, payment method, and date of payment.

3.1.5 Custodian Booking Management

FR-022

The system shall display a dedicated custodian dashboard showing all bookings for their managed hostel, categorised by status: Pending, Approved, Rejected.

FR-023

The system shall allow the custodian to view the full details of any booking request, including the student's submitted documents and payment confirmation status.

FR-024

The system shall allow the custodian to approve a pending booking by selecting and assigning a specific room number from the available inventory.

FR-025

The system shall allow the custodian to reject a pending booking and optionally provide a rejection reason.

FR-026

The system shall automatically update the booking status in the database to Approved or Rejected upon the custodian's decision, and notify the student via the in-app dashboard.

FR-027

The system shall automatically update the room status from Available to Booked when a booking is approved and a room number is assigned.

3.1.6 Room Inventory Management

FR-028

The system shall provide custodians with a visual interactive floor plan displaying each room's current status using colour-coded indicators: green (Available), yellow (Partially Booked), red (Fully Booked).

FR-029

The system shall allow custodians to add new rooms to the floor plan by specifying: block name, floor number, room number, room type (Single/Double/Self-Contained), and price per semester.

FR-030

The system shall allow custodians to update room details or mark a room as unavailable for maintenance.

3.1.7 Student Dashboard and Maintenance

FR-031

The system shall provide each student with a personal dashboard displaying their booking history, current booking status, assigned room (if approved), and payment records.

FR-032

The system shall allow a student to cancel a pending booking before it has been approved by the custodian.

FR-033

The system shall allow a student to submit a maintenance request for their assigned room by specifying the issue type and a description.

FR-034

The system shall allow custodians to view all maintenance requests for their hostel and update the resolution status of each request.

3.2 Non-Functional Requirements

3.2.1 Performance

NFR-001

The system shall return API responses for hostel listing and search queries within 2 seconds under normal network conditions.

NFR-002

The system shall serve the React frontend initial bundle in under 3 seconds on a standard 4G mobile connection, achieved through Vite's optimised production build and code splitting.

NFR-003

The system shall support at least 200 concurrent authenticated users without measurable degradation in API response time.

3.2.2 Security

NFR-004

The system shall enforce HTTPS on all frontend and backend Render.com services; HTTP connections shall be automatically redirected to HTTPS.

NFR-005

The system shall implement CORS protection on the API server, whitelisting only the deployed CLIENT_URL; all other origins shall receive an HTTP 403 Forbidden response.

NFR-006

The system shall never store, log, or transmit plain-text passwords at any point in the system lifecycle.

NFR-007

The system shall store all sensitive configuration values (MONGO_URI, JWT_SECRET, CLIENT_URL) exclusively as environment variables injected at runtime; they shall never be committed to version control.

NFR-008

The system shall sanitise all user inputs on the backend to prevent NoSQL injection attacks.

3.2.3 Availability and Reliability

NFR-009

The system shall maintain a system availability of at least 99% per calendar month, supported by Render.com's auto-restart policy and MongoDB Atlas managed uptime SLAs.

NFR-010

The system shall automatically restart the Node.js API server upon unexpected crashes without manual intervention.

NFR-011

The system shall preserve all booking and payment records in the event of a server restart; no data shall be stored exclusively in server memory.

3.2.4 Usability

NFR-012

The system shall implement a mobile-first responsive design that renders correctly on viewport widths from 320px (small smartphone) to 1920px (desktop widescreen).

NFR-013

The system shall provide toast notification feedback to users within 1 second of any significant action (booking submitted, payment processed, error encountered).

NFR-014

The system shall complete the entire booking flow within 5 user interactions in a single session without requiring the user to revisit previous pages.

3.2.5 Maintainability

NFR-015

The system shall organise the backend codebase using the MVC design pattern with separate directories for models, controllers, routes, and middleware.

NFR-016

The system shall organise the frontend codebase into feature-based modules (auth, home, hostels, booking, custodian, dashboard) with reusable components.

NFR-017

The system shall include a README.md in both the client and server repositories documenting environment setup, dependencies, and deployment instructions

3.3 External Interface Requirements

3.3.1 User Interfaces

The system shall provide the following key screens, consistent with the wireframes .

Screen	Primary Users	Key Elements
Home / Landing Page	All (public)	Hero section, popular hostels grid, search bar, navigation
Hostel Search Page	Students	Filter sidebar (location, price, gender), hostel listing cards
Hostel Detail Page	Students	Photos, amenities, pricing, Google Maps, room types, Book Now button

Booking Form Page	Students	Multi-step form: personal info, next-of-kin, health, document upload
Payment Page	Students	Payment method selection, booking summary, Complete Payment button
Student Dashboard	Students	Booking history, status badges, receipt download, maintenance request form
Custodian Dashboard	Custodians	Pending bookings queue, approve/reject controls, room floor plan

3.3.2 Software Interfaces

The system shall interface with the following software components:

Interface	Type	Protocol / Library	Purpose
MongoDB Atlas	Database	Mongoose ODM / TCP	Persistent data storage
Payment Gateway	External API	HTTPS / REST	Process Mobile Money and card payments
Google Maps API	External API	HTTPS/JavaScript SDK	Interactive hostel location maps
jsPDF + html2canvas	JavaScript Library	Browser-native	Client-side PDF receipt generation
Axios HTTP Client	Library	HTTPS / REST	Frontend-to-backend API communication

3.4 System Constraints

- The frontend must be built as a Single Page Application (SPA) using React 19 and Vite.
- The backend API must be stateless; all session state must be managed on the client using JWTs.
- The database must use MongoDB Atlas; relational databases are not permitted.
- The system must be deployed as two independent Render.com services: one static site (frontend) and one web service (backend).
- All environment variables must be configured via Render.com's environment variable settings; hardcoded secrets are prohibited.

- The system must not store credit card numbers or Mobile Money PINs; all sensitive payment data must be handled exclusively by the payment gateway provider.

4 Appendices

4.1 Appendix A - Glossary

This glossary supplements Section 1.3 with additional domain-specific terms:

Term	Meaning
Pending Approval	A booking status indicating the student has submitted and paid but the custodian has not yet reviewed it.
Confirmed	A booking status indicating the custodian has approved the booking and assigned a room number.
Rejected	A booking status indicating the custodian has declined the booking request.
Semester	An academic term at Makerere University; the standard billing period for hostel accommodation.
Floor Plan	A visual grid interface showing all rooms in a hostel block with their real-time occupancy status.
Booking Token	A unique alphanumeric identifier automatically generated for each booking record at creation.
Security Fee	A refundable deposit charged alongside the room fee, held by the custodian for the duration of stay.
Self-Contained	A room type with private bathroom facilities included within the room unit.
protect middleware	A Node.js/Express middleware function that validates the JWT before allowing access to a protected route.
seeding	The process of populating the MongoDB database with initial hostel and room data using migration scripts.

4.2 Appendix B -Use Case Summary

The following table summarises the primary use cases that drove the requirements in Section 3:

UC ID	Use Case	Actor	Related FRs
UC-01	Register / Login	Student, Custodian	FR-001 to FR-007
UC-02	Search and Filter Hostels	Student	FR-008 to FR-011
UC-03	Book a Room	Student	FR-012 to FR-016

UC-04	Make Payment and Receive Receipt	Student	FR-017 to FR-021
UC-05	Manage Booking Requests	Custodian	FR-022 to FR-027
UC-06	Manage Room Inventory	Custodian	FR-028 to FR-030
UC-07	Track Bookings and Submit Maintenance	Student	FR-031 to FR-034
UC-08	Resolve Maintenance Requests	Custodian	FR-034